

Racket

A short introduction. (K. Kirby)



Just as French and Spanish are descendants of Latin, **Racket** is a descendent of the ancient programming language *Lisp* from the late 1950s. One of the later dialects of Lisp was named *Scheme*, and Scheme evolved into Racket.

The first few sections below (I-XII) are just meant for reading and thinking, without experimenting on the computer.

Racket is a language for the description of patterns and their transformations. We call concrete realizations of patterns *expressions*.

I. ATOMS

The basic building blocks of expressions are **atoms**. There are a few kinds of atoms. We'll use these kinds:

Symbols, which are spelled out with letters, digits, and dashes, like:

`a b x y abulafia covid-19z lingua-characteristica`

Numbers, which can be whole numbers, numbers with decimal points, or fractions, like:

`14 -246 1.230 7/13`

Strings, which are pretty much anything between quotes, like:

`"dog" "covid-19z" "language is a virus from outer space"`

Finally, there is a special atom, the “null atom,” which is written like this:

`null`

II. LISTS OF ATOMS

Atoms are combined to make expressions by placing them in linear sequences called **lists**. A list is surrounded by parentheses. Here are some examples:

`(a b c a a)
(abulafia 17 4/51 "dog")
(x)
()`

The last list is empty, and it is regarded as equivalent to the atom `null`.

III. LISTS IN GENERAL

Expressions need not be just lists of atoms. Lists can contain lists too.

```
(a (b c) (d e f))  
((a b (c d (d (d) e)) e) 13 f)  
(( ) ((( ) ( ) ( ))) ( )))
```

Always, the parentheses have to be matched. For example, `(( ( ( ( ( )` is not a list, since the parentheses don't match up.

In the end, expressions can be either atoms or lists.

IV. LISTS CAN REPRESENT STRUCTURED KNOWLEDGE

A good language for representing knowledge should clearly display structure. Humans often structure complicated knowledge hierarchically. Nested lists can express hierarchies.

Here is one of many imaginable ways a data record could be represented as a list in Racket:

```
((name "delany" "samuel r") (books ("Babel-17" 1966) ("Nova" 1968) ("Dhalgren" 1974)))
```

Here is one of many imaginable ways the grammatical structure of an English sentence “**The dog ate my homework**” might be represented as a list in Racket:

```
(sentence  
  (form "the dog ate my homework")  
  (subject (attributes (person 3) singular animate (not human))  
    (form (noun-phrase  
      (article definite (form "the"))  
      (noun singular (form "dog")))))  
  (predicate (attributes (tense past) (person 3) singular)  
    (form (verb-phrase  
      (verb (form "eat" (past "ate")))  
      (object (form (noun-phrase  
        (adjective (possessive (person 1) singular (form "my"))  
        (noun singular (form "homework")))))))))
```

The indenting above is not part of the Racket language. It is just easier for humans to understand when it is printed prettily. Racket doesn't care if you write the list like this:

```
(sentence (form "the dog ate my homework") (subject (attributes (person 3) singular animate (not human)) (form (noun-phrase (article definite (form "the")) (noun singular (form "dog"))))) (predicate (attributes (tense past) (person 3) singular) (form (verb-phrase) (verb (form "eat" (past "ate")))) (object (form (noun-phrase (adjective (possessive (person 1) singular (form "my")) (noun singular (form "homework"))))))))
```

but writing this way is a pretty bad idea. The point is: lists are nice ways to represent structured knowledge.

V. PASSIVE AND ACTIVE EXPRESSIONS

Expressions can be passive or active.

A *passive* expression is something to be looked at, broken apart, or combined with other expressions. When an expression is to be treated as passive, you put a single quote mark in front of it.

`'(sqrt 49)`

Active expressions transform themselves into other expressions, using their first item as a kind of command. For example, when you remove the quotation mark from the expression above, it changes from passive to active, and turns itself into the square root of 49, which is 7. We write it this way:

`(sqrt 49)`
 $\Rightarrow 7$

This is a spontaneous transformation of one expression into another expression. Here the expression `(sqrt 49)` is transformed into the expression `7`. The atom `sqrt` gets treated as a command, called a procedure, which computes the square root of the number following it.

Expressions transform themselves from the inside out. For example

`(sqrt (sqrt (sqrt 256)))`
 $\Rightarrow$ `(sqrt (sqrt 16))`
 $\Rightarrow$ `(sqrt 4)`
 $\Rightarrow$ `2`

Expressions, when active, try to keep transforming themselves until they can't transform any more. Again, putting a quote in front of an expression renders it passive:

`'(sqrt (sqrt (sqrt 256)))`

This expression has been rendered passive by the quote, and will *not* turn itself into `2`.

A system of **blind thought** works by having (usually extremely complicated) expressions spontaneously transform into other expressions. If Leibniz is correct, all reasoning can be done this way. That's why Lisp, the ancestor of Scheme, was taken to be *the* language of artificial intelligence from about 1960 to 1985.

VI. INTRODUCING PROCEDURES: **first**

We have seen that the atom `sqrt` when placed at the beginning of an active expression causes a square root to be calculated. One says the atom `sqrt` denotes a **procedure**. More casually, we can say that `sqrt` "is" a procedure.

In fact, we don't care very much for square roots in this course. It's not a math course. We would like to have procedures that transform more general expressions, not just numbers.

The first procedure we will introduce is named **first**. It yields the first item in any list:

```
(first '(a b c))
```

```
⇒ 'a
```

```
(first '(a))
```

```
⇒ 'a
```

```
(first '((a b) (c d) e ((f))))
```

```
⇒ '(a b)
```

...note that it is *not* 'a

```
(first '())
```

....is an error, since () has no items at all

```
(first 'a)
```

....is an error since 'a is not a list

Notice the quote marks here. They allow **first** to operate on a passive expression. Look at these two examples very carefully:

```
(first '(first ((a b) c d)))
```

```
⇒ 'first
```

```
(first (first '(a b) c d)))
```

```
⇒ (first '(a b))
```

```
⇒ 'a
```

This kind of absolute precision is the first thing that makes dealing with code hard for humans.

VII. rest

Complementary to **first**, there is a procedure named **rest**. If you apply **rest** to a list, it yields the list with the first item removed:

```
(rest '(a b c))
```

```
⇒ '(b c)
```

```
(rest '((a b) (c d)))
```

```
⇒ '((c d))
```

```
(rest '())
```

```
⇒ '()
```

VIII. cons

We have two procedures that split expressions apart; now we introduce one that constructs a new expression from two pieces. It is named **cons**. The two pieces are precisely the **first** and the **rest** of the expression to be built:

```
(cons 'a '(b c))  
⇒ '(a b c)
```

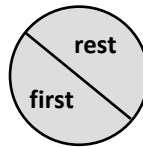
```
(cons 'a '())  
⇒ '(a)
```

```
(cons '(a) '(b))  
⇒ '((a) b) .  
...note that it is not '(a b)
```

IX. Understanding cons

*You should read this section eventually, but you could wait until you finish the rest of the document if you like. It's important, and logically fits right here, right after introducing **cons**. But we don't need it until later, so if it's confusing for now just skip to the next section and come back to it later.*

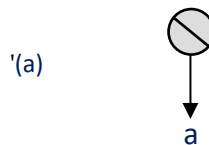
Suppose we introduce a little symbol called a **pair**:



We will omit the labels and draw it like this:



The list **'(a)** can be represented this way using one pair:

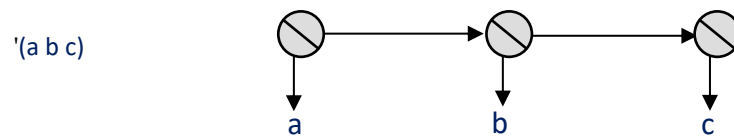


The **first** part of the cell is the atom **a**, and the rest part of the cell is **null** (and so left empty in the diagram), as one would expect:

```
(first '(a))  
⇒ 'a
```

```
(rest '(a))  
⇒ '()
```

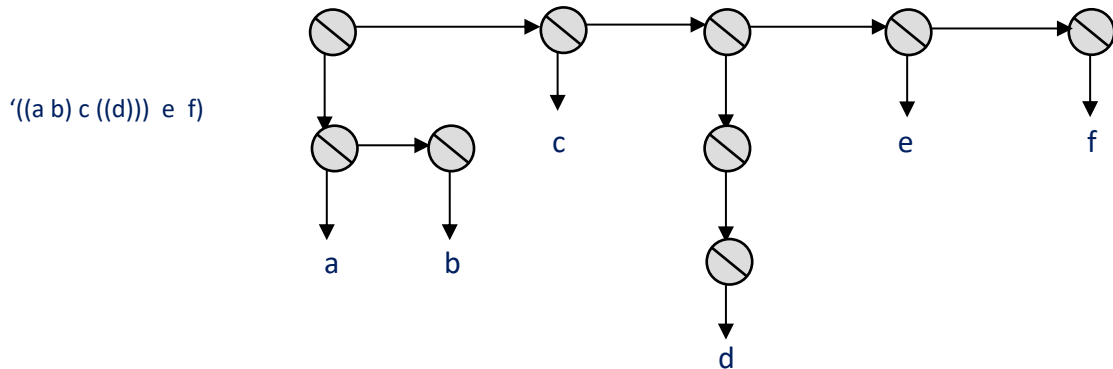
Here's a longer list that consists of three pairs:



The first pair on the left is said to “hold” the expression ‘(a b c)’. The pair in the middle holds the expression ‘(b c)’. The pair on the right holds the expression ‘(c)’. To find the **first** of the expression, we go down; to find the **rest**, we go right. Note that the **rest** of the last pair is **null**.

In this way, we can view every expression as a complex of just two things: *zero or more pairs* of *zero or more atoms*.

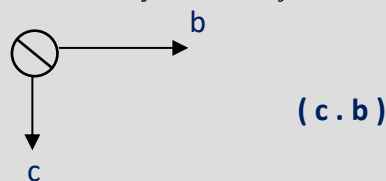
Here's a more complicated example with 9 pairs:



As an exercise, write out the expression corresponding to each pair in the diagram here.

Notice that the **first** of every pair in a list is never **null**. The **rest** is never an atom.

If a **rest** is an atom, it is not really an error, it just results in a special structure called a **dotted pair**:



We will not deal with dotted pairs in this course. I'm mentioning just in case you run into it by accident and wonder what it means.

X. MORE PROCEDURES

Here are some useful additional procedures that transform lists, shown by example:

The `list` procedure combines all the items that follow it into a single list. (Yes, this is useful, we will see.)

```
(list 'a 'x '(woof))  
⇒ 'a x (woof)
```

```
(list '(a) '(b c) 'd 'e)  
⇒ ' ((a) (b c) d e )
```

The `append` procedure takes a whole bunch of lists and glues them together:

```
(append '(a b) '(c d e))  
⇒ ' (a b c d e)
```

```
(append '(a) '(b (c)) '(e f))  
⇒ ' (a b (c) e f)
```

Make sure you see how `cons`, `list`, and `append` are different. For example:

```
(cons '(a) '(b))  
⇒ ' ((a) b)  
... think of first/rest !
```

```
(list '(a) '(b))  
⇒ ' ((a) (b))
```

```
(append '(a) '(b))  
⇒ ' (a b)
```

The `length` procedure tells you how many items are in a list:

```
(length '(a b c d e f g h i j))  
⇒ 10
```

```
(length '(a b (c d e f g h i) j))  
⇒ 4
```

And the `reverse` procedure does, well, you know...

```
(reverse '(a b c d))  
⇒ '(d c b a)
```

```
(reverse '(a (b c d)))  
⇒ ' ((b c d) a)
```

XI. PROCEDURES THAT ASK TRUE/FALSE QUESTIONS

There are two other atoms in the Racket language: `#t` (“true”) and `#f` (“false”). Here are some procedures that ask questions and yield true/false answers. Note that the “?” mark is part of the symbol, and is a giveaway that the procedure asks a true/false question:

```
(null? '(a b c))  
⇒ #f
```

```
(null? '())  
⇒ #t
```

```
(pair? '(a b c))  
⇒ #t
```

```
(pair? 'a)  
⇒ #f
```

```
(equal? '(a b c) '(a b c))  
⇒ #t
```

XII. PROCEDURES THAT DO ARITHMETIC

Scheme can do arithmetic too, of course. The asterisk is multiplication. You use `=`, `>` and `<` for comparing numbers.

```
(+ 40 12 100)  
⇒ 152
```

```
(- 15 5)  
⇒ 10
```

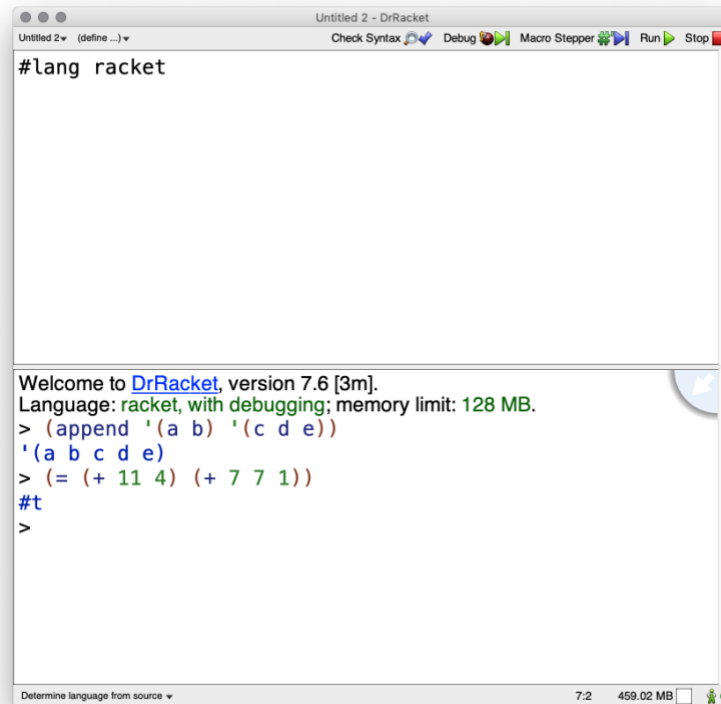
```
(* 3/4 1/2)  
⇒ 3/8
```

```
(= (+ 11 4) (+ 7 7 1))  
⇒ #t
```

```
(> 100 4)  
⇒ #t
```

```
(< 100 4)  
⇒ #f
```

At this point, it's time to download and install **DrRacket** and experiment with the Racket language. Go to racket-lang.org and select Download. It's available for PCs, Macs, almost anything. We'll have some videos on the Canvas site to help you get used to using it.

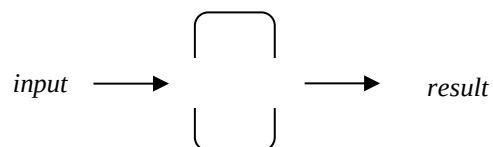


XIII. λ

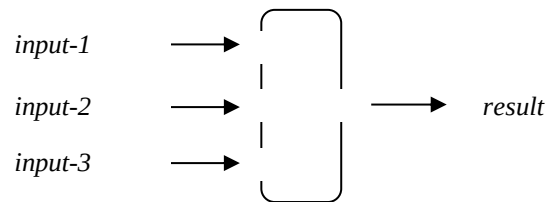
Starting now it gets deep.

The procedures supplied by Racket are very basic. The goal writing Racket code is to develop complex expressions that transform themselves and other expressions that reveal new knowledge. So we need to be able to define our own procedures.

A procedure looks something like this:



An expression goes in on the left, it is transformed, and a new expression comes out on the right. A procedure could take two, three or more original patterns and combine them to produce a transformed output:



The way we specify procedures is through *lambdas*. The lambda pattern looks like this:

`( λ list of inputs goes here result goes here )`

For example,

`( λ (x) (cons 'a x) )`

is a procedure that attaches the atom 'a onto the beginning of a list:

`(( λ (x) (cons 'a x)) '(p q r))`
 $\Rightarrow$ `'(a p q r)`

`(( λ (x) (cons 'a x)) '())`
 $\Rightarrow$ `'(a)`

You can spell out λ if you like:

`(lambda (x) (cons 'a x))`

but to get the cool Greek letter you hit the Command key plus 'V' in DrRacket.

Here is a procedure that takes two inputs:

`( λ (x y) (cons y x) )`

And here is how it is used:

`(( λ (x y) (cons y x)) '(a b) '(c d))`
 $\Rightarrow$ `'((c d) a b)`

XIV. DEFINITIONS

To make any progress whatsoever, we have to start making definitions:

```
(define s '(a (b c) d) )  
(define last ( λ (x) (first (reverse x)) ) )
```

A definition associates an atom with an expression or a procedure. We can now use the definitions in future expressions:

```
(first s)  
⇒ 'a
```

```
(last '(a b c))  
⇒ 'c
```

```
(last s)  
⇒ 'd
```

```
(equal? (first s) 'a)  
⇒ #t
```

Some practice: Given these two definitions:

```
(define langs '(popti mam kiche chol))  
(define cities '((rome (italy new-york)) (detroit (michigan)) (bellona ())))
```

show how the following fourteen expressions transform:

(length langs)	(length cities)
(first langs)	(first cities)
(rest langs)	(rest cities)
(rest (rest langs))	(rest (rest cities))
(first (rest langs))	(first (first langs))
(first (rest cities))	(cons (first cities) (rest cities))
(cons (first langs) (rest cities))	(cons '(first cities) (rest cities))

Run it on your own brain before you run it on the computer!

XV. IF

Procedures can only be useful if they can be conditional: we want to do something *if* something else happens; otherwise we do something different. You use the symbol **if** to do this in Racket:

```
(if (= 4 4) 'smile 'frown)
⇒ 'smile
```

```
(if (= 5 4) 'smile 'frown)
⇒ 'frown
```

If the condition is true, the result is the first choice. If the condition is false, the result is the second choice.

Here is how we could use **if** in a definition. Notice how we use indentation to make the definition clearer.

```
(define size
  (λ (x)
    (if (> x 1000)
        'big
        'small)))

(size 500)      ⇒ 'small
(size 1500)     ⇒ 'big
```

We can use **and**, **or** and **not** to help us define conditional transformations. For example:

```
(define grade
  (λ (x)
    (if (or (equal? x 'a) (equal? x 'b))
        'good
        'not-so-good)))

(grade 'a) ⇒ 'good
(grade 'c) ⇒ 'not-so-good
```

XVI. SELF REFERENCE

A definition of a procedure can refer to that same procedure in some cases:

```
(define stutter
  (λ (s)
    (if (null? s)
        null
        (cons (first s)
              (cons 'um (stutter (rest s)))))))
```

Think carefully how this could work. For example:

`(stutter '(a b c))` $\Rightarrow$ `(a um b um c um)`

The phenomenon of self-reference is one of the deepest concepts in computer science. It is called **recursion**. It can be used to establish limits on what is computable at all.

XVII. Practice

What do the following procedures do?

```
(define mystery1
  (λ (s)
    (cons (length s) s)))

(define mystery2
  (λ (s)
    (cons 'xx (rest s))))

(define mystery3
  (λ (x y)
    (list (first x) (first y))))

(define mystery4
  (λ (x)
    (if (> (length x) 1)
        (cons (first x) (rest (rest x)))
        x)))

(define mystery5
  (λ (x)
    (if (> (length x) 1)
        (cons (first x) (mystery5 (rest (rest x))))
        x)))

(define mystery6
  (λ (x)
    (if (null? x)
        0
        (+ (first x) (mystery6 (rest x))))))
```

Given the following definition:

```
(define langs
  '(popti mam kiche chol))
```

Try these:

```
(mystery1 '(cat dog))
(mystery1 null)

(mystery2 langs)

(mystery3 langs '(cat dog))

(mystery4 null)
(mystery4 '(a))
(mystery4 '(a b))
(mystery4 '(a b c d e f g))

(mystery5 '(a b c d e f g))

(mystery6 null)
(mystery6 '(4))
(mystery6 '(3 5 7))
```

3. Define a procedure that tells whether the first two elements of a list are the same.

```
(same-start? '(a a b c)) ⇒ #t
```

```
(same-start? '(a b b c)) ⇒ #f
```

4*. Define a procedure that takes two lists of equal length and interleaves them.

```
(interleave '(a b c) '(d e f)) ⇒ '(a d b e c f)
```

* Kind of hard. Use `interleave` in the definition of `interleave`, as in `stutter`, above.